StoryVox — Product Documentation

1. Product overview

StoryVox is a local-first multilingual storytelling application. A user enters a story idea, chooses a language, style and length, and StoryVox writes an original story with a pretrained language model and narrates it with a pretrained multilingual speech model. Stories, narration audio and evaluation results are stored locally in SQLite and on disk.

2. Key features

Story creator with language/style/length selection; automatic narration with playback and download; regenerate-narration action; searchable history with view/play/download/delete; measurable evaluation dashboard (WER, CER, similarity, latency, RTF); lazy model loading with CPU support; local storage and privacy note.

3. User workflow

1) Enter a story idea. 2) Select language, style and length. 3) Press Generate Story. 4) Read the story, play or download the narration. 5) Revisit or delete items in History. 6) Run and review Evaluation for measured pipeline quality.

4. Technology stack

Python, FastAPI, Uvicorn, SQLite + SQLAlchemy, Pydantic, Hugging Face Transformers + PyTorch (Qwen3 story models, MMS multilingual TTS checkpoints, Whisper ASR), soundfile/numpy audio handling, HTML/CSS/JavaScript frontend served by FastAPI, pytest for tests.

5. Architecture summary

Single FastAPI service on 127.0.0.1. Layers: REST API -> services (story_generator, tts_service behind a TTSProvider interface, asr_service, evaluation_service) -> SQLite persistence + WAV files under data/audio. Models load lazily and are cached in memory; CUDA is used when available, otherwise CPU.

6. Story-generation pipeline

The user prompt is combined with target language, style guidance and length budget into an instruction prompt. A pretrained Qwen3 instruction model generates a title plus story body (temperature/top-p/max-tokens configurable). Output is parsed into title/body, word counts and timings are recorded, and the record is stored in the stories table.

7. Multilingual TTS pipeline

Story text is chunked at sentence boundaries, synthesized per chunk with the language-matched MMS checkpoint (VITS), normalized to 16 kHz mono, concatenated with short pauses, validated for non-silence, and written as WAV. Synthesis time, duration and RTF (synthesis_time / audio_duration) are stored in audio_outputs. An offline OS speech fallback is used only when the pretrained checkpoint cannot load, and the actual model id is always recorded.

8. Database design

stories(id, prompt, title, story_text, language_code, language_name, style, length_option, word_count, created_at, generation_time_seconds); audio_outputs(id, story_id, file_path relative, duration_seconds, sample_rate, tts_model, synthesis_time_seconds, rtf, created_at); evaluations(id, evaluation_type, language_code, sample_count, metrics_json, created_at). Deleting a story removes its rows and audio files.

9. Evaluation methodology

scripts/evaluate.py generates stories from fixed prompts, synthesizes narration, transcribes it with Whisper, and computes WER/CER against the story text, prompt-story similarity (multilingual sentence model when available, otherwise TF-IDF cosine), ROUGE-L overlap, plus story/TTS latency and RTF. Results are saved as JSON and CSV under data/evaluations and shown in the UI. No values are pre-filled; unevaluated states display 'Evaluation has not been run yet.'

10. Actual results/metrics

Latest evaluation file: storyvox_evaluation_20260908_104144.json | samples: 2 | WER: None | CER: None | similarity: 0.8018 | story latency: 0.0s | TTS latency: 0.225s | RTF: 0.0056

11. How to run

Method A: double-click run.bat, wait for dependency install and first-time model downloads, open http://localhost:8000. Method B: Docker via run-docker.bat. Method C: create a venv, pip install -r requirements.txt, run uvicorn app.main:app --host 127.0.0.1 --port 8000. Full steps are in HOW_TO_RUN.md.

12. Limitations

CPU inference is slower than GPU; first runs download model weights; narration quality depends on the selected checkpoint and language; offline fallback voices are more robotic; very long stories are capped by token budgets; evaluation transcription adds runtime cost.

13. Future improvements

Streaming generation progress, per-story voice selection where the TTS model supports it, background evaluation jobs, export of stories to e-book formats, and optional objective naturalness scoring when a lightweight compatible model is available.